Time Complexity

Time complexity in computer science describes how the execution time of an algorithm grows as the input size increases. It essentially quantifies the number of operations an algorithm performs as a function of the input data size.

Example

```
for i in range(n):
  print(i)
```

→ n operations

The loop runs n times.

Runtime vs. input size

n = 5 → 10 msec

×10

n = 50 → 100 msec

Asymptotic Notations

Ω Best Case

Θ Average Case

O Worst Case

```
for i in range(n):
  if i==x:
    break
  print(i,end=" ")
```

x = 0 → best case · x = n → worst case

Case	Operations	Notation
Best Case	1 operation	Ω(1)
Worst Case	n operations	O(n)
Average Case	n/2 operations	Θ(n/2)

Types of Time Complexities

Time complexities are categorized based on how an algorithm's runtime scales with the input size.

Common types include:

Constant	Logarithmic	Linear	Quadratic	Exponential
O(1)	O(log n)	O(n)	O(n^2)	O(2^n)

Time Complexity Examples

Linear Time (O(n))

```
for i in range(n):
  print(i,end=" ")
```

Constant Time (O(1))

```
for i in range(10):
  print(i,end=" ")
```

Quadratic Time (O(n^2))

```
for i in range(n**2):
  print(i,end=" ")
```

Logarithmic Time (O(log n))

```
n = 100
while n>0:
  print(n,end=" ")
  n//=2
```

Output

```
100 50 25 12 6 3 1
```

```
n = 1
while n⩽100:
  print(n,end=" ")
  n*=2
```

Other Time Complexities

Linearithmic Time

O(n log n)

Cubic Time

O(n^3)

Factorial Time

O(n!)

Example code

```
i = 1
n=100

for j in range(n):
  while i⩽n:
    print(i,end=" ")
    i*=2
```

Comparison of Time Complexities

O(1) < O(log n) < O(n) < O(n log n) < O(n^2) < O(2^n) < O(n!)

Space Complexity

Space complexity refers to the amount of memory an algorithm uses as a function of the input size. It's a measure of how much memory an algorithm requires to execute and solve a problem.

```
n = 100
m = 10

list1 = [0]*10

for i in range(n):
  list1.append(m)
```